

Full Build & Run Guide: Docker & Project Setup

Here is a complete summary of all the steps, files, and commands required to build your Docker image, compile your C++ code, and run your Python application.

Part 1: Building the Docker Image (On Your Host Machine)

This is the final, corrected `Dockerfile` that includes all the fixes for package names (`libdc1394-dev` , `qtbase5-dev`), accepts the Conda Terms of Service, and correctly sets up the `my` conda environment.

Final Dockerfile

```
FROM nvidia/cuda:12.8.0-cudnn-runtime-ubuntu22.04

# Set the timezone
ENV TZ=US/Pacific
RUN ln -snf /usr/share/zoneinfo/$TZ /etc/localtime && echo $TZ > /etc/timezone

# Update and install essential libraries and tools (with Ubuntu 22.04 corrections)
RUN apt-get update --fix-missing && \
    apt-get install -y \
        libgtk2.0-dev \
        wget bzip2 ca-certificates curl git vim tmux g++ gcc build-essential cmake checkinstall \
        libjpeg8-dev libtiff5-dev pkg-config yasm \
        libavcodec-dev libavformat-dev libswscale-dev \
        libdc1394-dev \
        libxine2-dev libv4l-dev \
        qtbase5-dev \
        libtbb-dev libatlas-base-dev libfaac-dev libmp3lame-dev libtheora-dev libvorbis-dev \
        libopencore-amrnb-dev libopencore-amrwb-dev \
        x264 v4l-utils \
        libprotobuf-dev protobuf-compiler libgoogle-glog-dev libgflags-dev libgphoto2-dev lib \
        doxygen libflann-dev libboost-all-dev proj-data libproj-dev libyaml-cpp-dev \
        cmake-curses-gui libzmq3-dev freeglut3-dev && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/*

# Install pybind11
RUN cd / && git clone [https://github.com/pybind/pybind11](https://github.com/pybind/pybind11) && \
    cd pybind11 && git checkout v2.10.0 && \
    mkdir build && cd build && cmake .. -DCMAKE_BUILD_TYPE=Release -DPYBIND11_INSTALL=ON \
    make -j6 && make install

# Install Eigen
RUN cd / && wget [https://gitlab.com/libeigen/eigen/-/archive/3.4.0/eigen-3.4.0.tar.gz](https://gitlab.com/libeigen/eigen/-/archive/3.4.0/eigen-3.4.0.tar.gz) && \
    tar xvfz ./eigen-3.4.0.tar.gz && \
    cd eigen-3.4.0 && \
    mkdir build && \
    cd build && \
    cmake .. && \
    make install

# Switch to bash shell for conda setup
SHELL ["/bin/bash", "--login", "-c"]

# Install Miniconda
RUN cd / && wget --quiet [https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86](https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86) \
    /bin/bash /miniconda.sh -b -p /opt/conda && \
    # --- FIX 1: Accept Conda Terms of Service ---
```

```

/opt/conda/bin/conda tos accept --override-channels --channel [https://repo.anaconda.
/opt/conda/bin/conda tos accept --override-channels --channel [https://repo.anaconda.
# --- FIX 2: Correctly initialize conda env ---
ln -s /opt/conda/etc/profile.d/conda.sh /etc/profile.d/conda.sh && \
echo ". /opt/conda/etc/profile.d/conda.sh" >> ~/.bashrc && \
/bin/bash -c "source ~/.bashrc" && \
/opt/conda/bin/conda update -n base -c defaults conda -y && \
/opt/conda/bin/conda create -n my python=3.8 && \
conda init bash && \
echo "conda activate my" >> ~/.bashrc

# Add conda to PATH (Corrected Syntax)
ENV PATH="/opt/conda/envs/my/bin:$PATH"

# Activate conda environment and install Python packages
# The 'my' env is auto-activated by the SHELL directive sourcing .bashrc
RUN pip install torchvision==0.16.0+cu121 torchaudio==2.1.0 torch==2.1.0+cu121 --index-ur
    pip install "git+[https://github.com/facebookresearch/pytorch3d.git@stable](https://g
    pip install scipy joblib scikit-learn ruamel.yaml trimesh pyyaml opencv-python imagei

# Clone and install nvdiffrast
RUN cd / && git clone [https://github.com/NVlabs/nvdiffrast](https://github.com/NVlabs/nv
    cd /nvdiffrast && pip install .

# Set environment variable for OpenEXR support in OpenCV
ENV OPENCV_IO_ENABLE_OPENEXR=1

# Install additional Python packages
RUN pip install scikit-image meshcat webdataset omegaconf pypng roma seaborn opencv-contr
    conda install -y -c anaconda h5py

# Set shell to bash
ENV SHELL=/bin/bash
RUN ln -sf /bin/bash /bin/sh

```

Build Command

Save the file above as `Dockerfile` , open a terminal in the same directory, and run:

```
docker build -t your-image-name .
```

(Replace `your-image-name` with your preferred tag, e.g., `live-pose-env`).

Part 2: Running the Container (On Your Host Machine)

To run the container, you need to:

1. Share your project code directory (e.g., `FoundationPose`) into the container.
2. Enable GPU access.
3. **Share host devices so the camera can be found.**
4. Set the working directory.

```

# Replace /path/to/your/FoundationPose with the actual path on your computer
# Replace your-image-name with the tag you used in Part 1
#
# --- FIX: Add device access for RealSense Camera ---

```

```
# We add -v /dev:/dev to share the host's devices with the container,
# allowing pyrealsense2 to detect the connected camera.

docker run -it --gpus all \
  -v /path/to/your/FoundationPose:/app \
  -v /dev:/dev \
  -w /app \
  --rm \
  your-image-name
```

You will now be inside the container, at the `/app` prompt.

Part 3: Building C++ Code (Inside the Container)

This step uses the corrected `build.bash` and the `build` command with the correct path for Python 3.8.

`build.bash`

This file should be in your project directory (e.g., at `/app/build.bash`).

```
#!/bin/bash
DIR=$(pwd)

# Create build directory
cd $DIR/FoundationPose/mycpp/
mkdir -p build
cd build

# Run cmake with the correct python and make executables specified
# This fixes the "gmake" error
cmake .. \
  -DPYTHON_EXECUTABLE=$(which python) \
  -DCMAKE_MAKE_PROGRAM=$(which make)

# Run make
make -j11

cd ${DIR}
```

Build Command

Run this from the `/app` directory inside your container:

```
# This command points to the correct python3.8 path for pybind11
CMAKE_PREFIX_PATH=$CONDA_PREFIX/lib/python3.8/site-packages/pybind11/share/cmake/pybind11
```

Part 4: Running Your Python App (Inside the Container)

This section covers the final steps to activate the environment and run your code.

1. Activate Conda Environment

When you first enter the container, the conda environment may not be active. You only need to do this once per container session.

```
# This "reloads" your shell to make 'conda activate' work
source ~/.bashrc
```

Your shell prompt should change to `(my) root@...:/app#`, showing the `my` environment is active. If it doesn't, run `conda activate my`.

2. Final Package Fix (If Needed)

During our debugging, `pyrealsense2` failed to install correctly during the Docker build. If you still get a `ModuleNotFoundError`, run this one-time fix:

```
pip install pyrealsense2
```

3. Set PYTHONPATH for Custom C++ Module

You must tell Python where to find your compiled `mycpp` module.

```
# This adds the build directory to Python's search path
export PYTHONPATH=$PYTHONPATH:$(pwd)/FoundationPose/mycpp/build
```

4. Run Your Application

Now you can run your Python script, which will use the `python3.8` from the `my` environment and be able to find `mycpp`.

```
python run_live.py
```